



网络空间安全创新创业实践

实验 1 比特币交易与区块数据解析

刘育玮 202300460006 网安 23.1

2026 年 7 月 22 日

目录

1	实验环境	1
2	实验过程	1
2.1	准备实验环境	1
2.2	创建测试交易	1
2.3	准备数据解析基础函数	2
2.4	获取并解析交易数据	3
2.5	获取并解析区块数据	5
3	实验结果	9
4	附录	9

1 实验环境

操作系统: Ubuntu 22.04 LTS On WSL

Python 解释器: Python 3.10.12

Bitcoin 客户端: Bitcoin Core 31.0.0

2 实验过程

2.1 准备实验环境

安装 Bitcoin Core 客户端, 启动客户端并等待区块链数据完成同步, 如图 1 所示。

图 1: 启动客户端

```
> ~ bitcoin -testnet -daemon
Bitcoin Core starting
```

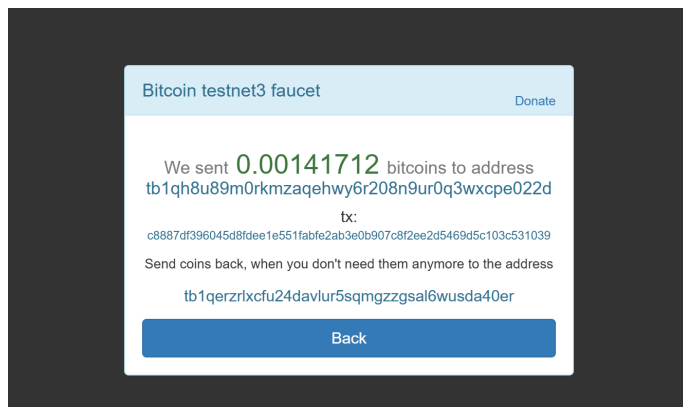
创建测试用比特币钱包 `testwallet`, 并获取一个地址用于接收测试比特币, 如图 2 所示, 获取到的地址为 `tb1qh8u89m0rkmaqhwy6r208n9ur0q3wxcpe022d`。

图 2: 创建测试钱包并获取地址

```
> ~ bitcoin-cli -testnet createwallet testwallet
{
  "name": "testwallet"
}
> ~ bitcoin-cli -testnet getnewaddress
tb1qh8u89m0rkmaqhwy6r208n9ur0q3wxcpe022d
```

从比特币测试网络获取测试比特币到该地址, 如图 3 所示。

图 3: 从测试网络获取测试比特币



等待交易确认后, 查询钱包余额, 如图 4 所示, 钱包余额为 0.00141712 BTC。

图 4: 查询钱包余额

```
> ~ bitcoin-cli -testnet getbalance
0.00141712
```

2.2 创建测试交易

使用第一次获取的地址发送测试比特币, 再获取一个地址用于接收测试比特币, 如图 5 所示, 获取到的地址为 `tb1q42p7e3uqus54fxze9g26rha88nqpplzumecua`。

图 5: 创建测试钱包并获取地址

```
➤ ~ bitcoin-cli -testnet getnewaddress  
tb1q42p7e3uqus54fxze9g26rha88nqpplzumeua
```

向新获取的地址发送 0.0001 的测试比特币，记录本次交易的 TXID，等待交易确认后，查询钱包余额，如图 6 所示，钱包余额为 0.00141569 BTC，扣除了本次交易的手续费。

图 6: 创建测试交易并查询余额

```
➤ ~ bitcoin-cli -testnet sendtoaddress tb1q42p7e3uqus54fxze9g26rha88nqpplzumeua 0.0001  
bb5ef9ab88c18759d4fb5fab31ef3c58ed62ab267aba338fd9effb2fdb03fb15  
➤ ~ bitcoin-cli -testnet getbalance  
0.00141569
```

2.3 准备数据解析基础函数

编写解析交易和区块数据所需的基础函数，如代码 1 所示。

代码 1: 解析交易和区块数据所需的基础函数代码

```
1 import struct  
2  
3 class Parser:  
4     # 初始化  
5     def __init__(self, raw):  
6         self.raw = bytes.fromhex(raw)  
7         self.offset = 0 # 当前解析位置  
8     # 读出指定长度的字节数据  
9     def read(self, n):  
10         data = self.raw[self.offset:self.offset + n] # 读取 n 个字节  
11         self.offset += n # 更新解析位置  
12         return data  
13     # 预览指定长度的字节数据  
14     def preview(self, n):  
15         data = self.raw[self.offset:self.offset + n] # 读取 n 个字节  
16         return data  
17     # 读取 32 位小端序无符号整数  
18     def readuint32(self):  
19         return struct.unpack("<I", self.read(4))[0]  
20     # 读取 64 位小端序无符号整数  
21     def readuint64(self):  
22         return struct.unpack("<Q", self.read(8))[0]  
23     # 读取可变长度小端序无符号整数  
24     def readvarint(self):  
25         prefix = self.read(1)[0]  
26         if prefix < 0xfd:  
27             # 直接表示的 8 位整数  
28             return prefix  
29         elif prefix == 0xfd:  
30             # 0xfd 后为 16 位整数
```

```

31         return struct.unpack("<H", self.read(2))[0]
32     elif prefix == 0xfe:
33         # 0xfe 后为 32 位整数
34         return struct.unpack("<I", self.read(4))[0]
35     else:
36         # 0xff 后为 64 位整数
37         return struct.unpack("<Q", self.read(8))[0]

```

将代码 1 保存为 `parse.py` 文件，供后续解析脚本调用。

2.4 获取并解析交易数据

获取本次测试交易的原始数据并保存到 `tx.hex` 文件中，如图 7 所示。

图 7: 获取测试交易的原始数据

```

> Lab-1 bitcoin-cli -testnet getrawtransaction bb5ef9ab88c18759d4fb5fab31ef3c58ed62ab267aba338fd9effb2fdb03fb15 | tee tx.hex
0200000000001013910533c105c9d46d5e22e8f7c900b3eabe2bffa51e5e1de8f5d0496f37d88c800000000fdffffff02f101020000000000160014c691ed
a4ba109a44a0da4894f5210f8fcc880a61102700000000000160014aa83ecc780e4295498592a15a1dfa73cc010dfe202473044022069e10cac88928f9edd
cc9044869498613c28b3381f2f7f78f5271c53606236000220277dbeff87be0d03f0af083d2ca1c6a86eeef26ca0a5d576b678385ee28913570121036567d9
953163c93d55a8d9ead08c4beacf017a5463d1cc336120b120ac90e88b32734d00

```

编写脚本解析 `tx.hex` 文件中的交易数据，如代码 2 所示。

代码 2: 解析测试交易数据的脚本代码

```

1  import parse
2  import json
3
4  class TxParser(parse.Parser):
5      # 解析交易数据
6      def parsetx(self):
7          tx = {}
8          # 版本号
9          tx["version"] = self.readuint32()
10         # SegWit 标记
11         segwit = False
12         if self.preview(2) == b'\x00\x01':
13             segwit = True
14             self.read(2) # 跳过标记字节
15         tx["segwit"] = segwit
16         # 输入
17         vin_count = self.readvarint() # 输入数量
18         tx["vin"] = []
19         for _ in range(vin_count):
20             vin = {}
21             # 输入 TXID
22             vin["txid"] = self.read(32)[::-1].hex() # 反转顺序
23             # 输入的输出生索引

```

```

24         vin["vout"] = self.readuint32()
25         # 输入脚本
26         vin["scriptSig"] = self.read(self.readvarint()).hex()
27         # 输入序列号
28         vin["sequence"] = self.readuint32()
29         # 记录输出信息
30         tx["vin"].append(vin)
31     # 输出
32     vout_count = self.readvarint() # 输出数量
33     tx["vout"] = []
34     for _ in range(vout_count):
35         vout = {}
36         # 输出金额
37         vout["value_satoshi"] = self.readuint64()
38         # 输出脚本
39         vout["scriptPubKey"] = self.read(self.readvarint()).hex()
40         # 记录输出信息
41         tx["vout"].append(vout)
42     # 输入的 Witness 数据
43     if segwit:
44         for vin in tx["vin"]: # 对每个输入
45             witness_count = self.readvarint() # Witness 元素数量
46             vin["txinwitness"] = []
47             for _ in range(witness_count):
48                 item_len = self.readvarint() # Witness 元素长度
49                 item = self.read(item_len) # Witness 元素数据
50                 vin["txinwitness"].append(item.hex())
51     # 锁定时间
52     tx["locktime"] = self.readuint32()
53     # 返回解析后交易数据
54     return tx
55
56 if __name__ == "__main__":
57     # 读取原始交易数据
58     with open("tx.hex", "r") as f:
59         rawtx = f.read().strip()
60     # 初始化解析器
61     parser = TxParser(rawtx)
62     # 解析交易数据
63     result = parser.parsetx()
64     # 输出解析后交易数据
65     print(json.dumps(result, indent=2))

```

将代码 2 保存为 `txparse.py` 文件并运行该文件解析交易数据，并将解析结果保存到 `txparse_result.txt` 文件中，如图 8 所示，输出了交易的各个字段信息。

图 8: 解析测试交易数据

```

➤ Lab-1 python txparse.py | tee txparse_result.txt
{
  "version": 2,
  "segwit": true,
  "vin": [
    {
      "txid": "c8887df396045d8fdee1e551fabfe2ab3e0b907c8f2ee2d5469d5c103c531039",
      "vout": 0,
      "scriptSig": "",
      "sequence": 4294967293,
      "txinwitness": [
        "3044022069e10cac88928f9eddc9044869498613c28b3381f2f7f78f5271c5360623600020277dbeff87be0d03f0af083d2ca1c6a86eeef26ca5a5d576b678385ee289135701",
        "036567d9953163c93d55a8d9ead08c4beacf017a5463d1cc336120b120ac90e88b"
      ]
    },
    {
      "value_satoshi": 131569,
      "scriptPubKey": "0014c691eda4ba109a44a0da4894f5210f8fcc888a61"
    },
    {
      "value_satoshi": 10000,
      "scriptPubKey": "0014aa83ecc780e4295498592a15a1dfa73cc010dfe2"
    }
  ],
  "locktime": 5075762
}

```

2.5 获取并解析区块数据

获取本次测试交易对应区块的哈希，再通过区块哈希获取该区块的原始数据并保存到 `block.hex` 文件中，如图 9 所示。

图 9: 获取测试交易对应区块的原始数据

[illegible]

编写脚本解析 `block.hex` 文件中的交易数据，如代码 3 所示。

代码 3: 解析测试交易对应区块数据的脚本代码

```
1 import parse
2 import hashlib
3 import json
4
5 class BlockParser(parse.Parser):
6     # 解析区块头数据
```

```

7 def parseheader(self):
8     header = {}
9     # 版本号
10    header["version"] = self.readuint32()
11    # 前一区块哈希
12    header["previousblockhash"] = self.read(32)[::-1].hex()
13    # Merkle 根哈希
14    header["merkleroot"] = self.read(32)[::-1].hex()
15    # 时间戳
16    header["time"] = self.readuint32()
17    # 难度目标
18    header["bits"] = self.readuint32()
19    # 随机数
20    header["nonce"] = self.readuint32()
21    # 返回解析后的区块头数据
22    return header
23 # 解析交易数据
24 def parsetxid(self):
25     # 版本号
26     start_ver = self.offset # 起始位置
27     self.readuint32()
28     stop_ver = self.offset # 结束位置
29     # SegWit 标记
30     segwit = False
31     if self.preview(2) == b'\x00\x01':
32         segwit = True
33         self.read(2) # 跳过标记字节
34     # 输入
35     start_io = self.offset # 起始位置
36     vin_count = self.readvarint() # 输入数量
37     for _ in range(vin_count):
38         # 输入 TXID
39         self.read(32)[::-1].hex() # 反转顺序
40         # 输入的输出索引
41         self.readuint32()
42         # 输入脚本
43         self.read(self.readvarint()).hex()
44         # 输入序列号
45         self.readuint32()
46     # 输出
47     vout_count = self.readvarint() # 输出数量
48     for _ in range(vout_count):
49         # 输出金额

```



```

50         self.readuint64()
51         # 输出脚本
52         self.read(self.readvarint()).hex()
53     stop_io = self.offset # 结束位置
54     # 输入的 Witness 数据
55     if segwit:
56         for _ in range(vin_count): # 对每个输入
57             witness_count = self.readvarint() # Witness 元素数量
58             for _ in range(witness_count):
59                 self.read(self.readvarint()).hex()
60         # 锁定时间
61         start_lktm = self.offset # 起始位置
62         self.readuint32()
63         stop_lktm = self.offset # 结束位置
64         # 返回解析后交易数据
65         txraw = self.raw[start_ver:stop_ver] + \
66                 self.raw[start_io:stop_io] + \
67                 self.raw[start_lktm:stop_lktm] # 获取交易原始数据
68         txid = hashlib.sha256(
69             hashlib.sha256(txraw).digest()
70             ).digest()[::-1].hex() # 计算 TXID
71         return txid
72     # 解析区块数据
73     def parseblock(self):
74         block = {}
75         # 区块头
76         block["header"] = self.parseheader()
77         # 交易数量
78         tx_count = self.readvarint()
79         block["tx_count"] = tx_count
80         # 交易
81         block["tx"] = []
82         for _ in range(tx_count):
83             txid = self.parsetxid()
84             block["tx"].append(txid)
85         # 返回解析后的区块数据
86         return block
87
88 if __name__ == "__main__":
89     # 读取原始区块数据
90     with open("block.hex", "r") as f:
91         rawblock = f.read().strip()
92     # 初始化解析器

```

```

93 parser = BlockParser(rawblock)
94 # 解析区块数据
95 result = parser.parseblock()
96 # 输出解析后区块数据
97 print(json.dumps(result, indent=2))

```

将代码 3 保存为 `blockparse.py` 文件并运行该文件解析区块数据，并将解析结果保存到 `blockparse_result.txt` 文件中，如图 10 所示，输出了区块的各个字段信息。

图 10: 解析测试交易对应区块数据

```

> Lab-1 python blockparse.py | tee blockparse_result.txt
{
  "header": {
    "version": 536870916,
    "previousblockhash": "00000000139b1a8cd29a5db67060142603368d839d1430784d451bfdd2b250da",
    "merkleroot": "6acb46b3912be4fd4647391c9860bf7ec085aa572c9baa2ecad498351e419338",
    "time": 1784599689,
    "bits": 486604799,
    "nonce": 434194875
  },
  "tx_count": 60,
  "tx": [
    "4a63569eabf541330deafa601cb598e6b3bb58ee641815719aee6616b826c156",
    "bc3b7b32358fe964b9f6b659ed9a282d78d69e59883a29743b0161fd3f0742ee",
    "5ef217e3eb7d39b27ae9d526233ee21f715b16e668f4615d68e132e0b670b565",
    "2bbd7857acdaf98bb0bcfc4d2ed5635c1939679e870a2047481a4344a90ffdc4",
    "482a42a831c3ed8d192a083b1c0190b8d981b8a49c4d916256c796cea4fd644",
    "df34d5e77648b2e71b9de25f08b64ede38d49c555132631316aa18bc3ad671f",
    "c664bad234d1f6b27fd4bcf64bc825010d943d8ec4435081541f785d1b31b2ac",
    "08fa8d530a127a6af1087fae1a4818a3cc9891c37b227e8084ef6d490eca02b",
    "637f1de269e6730caf043072ddd3063b51fdad581e4c70e98dada3545e3f4cd",
    "50b32f892256fd7bc38eb4a3f0d3b7bb8d3393055b95ef84cace93e0014d0ecc",
    "2cc9da4241864d7158396d8d95c08d942e9d485849327848b78ee60967105e67",
    "d13cf6942d5ff7b5d0a83a351f920029ada8421886e48513ed897c4dff0fe08",
    "e9ea51c31d54232ae06fbd5d7694c979508d527001fab64b6c065de2d4aa30b",
    "64f8762129940966526ea8854ea9b4fa56fcb76289a8e8917f7b28eff044731c",
    "5611c6b67cef313afabdec98ca2881498fb0887a47f4287fe7b29d8a579fa1d",
    "0f9f9392923ebd9ab2c4fee5acd136cf3eebac4d53855a14da4027d999364623",
    "90cf451b0ee4d9bee8f8ba7f3905d783f8ae3899aaa939a913d66e17c95cd024",
    "60bb8c408f642f710f8f041712efbc94f70fcd469d3c7c24b8f0e40b3cfcc29",
    "f1ffa5ea780971d197cd79c0a546b81d83f9ced503982436c1ad3925b0df4259",
    "737062086e1f326c7a6a19cd988f2edc2d0bd99fddf2cc827016a94a3e37459",
    "6baabe4dd9c57aef9a3cb24c0daa02071a3859b1706aef654ec3573f341a625b",
    "b17bd3cd21cca9fb15186d7dcd5fa5a41d7d841e599d5e2d2267da85fa49b35b",
    "d949de8ee8403d5883590d9cc48ab6bb36f5ac3c2c0bc4aac5ab2a3ed1b825c",
    "1dea77a90400a10e253f2232678d4d7d7f375c9c7473c1e2a0d2a9a337685b5f",
    "c424e3b0624a0f60d622b0623c690a409bb86b67484ad6b78053d6d41fb56",
    "8bf5e81ce36ca2ef54713ec67e065b6e2b996ddfc46461d09edc0032c528765",
    "3e5bdac1a1cc76cd2b25a800e69bf1f5652c85bd7820d860589d83529c2c6d",
    "70319b62d33b96a5fc38a32ba15464a03e57b695dd477da75007771070b06e",
    "23849f0e00b539fc20b52c4d984eec5875641b962fb752167a10823b5f3f96e",
    "5c8fb2785930b23eaa0fb9dad73d877d59e707aa259cab7123224cf3c3d71",
    "4b3f7e499459965b44b9aa72db620c868f10a931bd3ba0d321371e6338b27f",
    "8df9fcada91a379fd641866d959f8471e5c093d6dd57f6a6d5799821847e282",
    "4d87e120619ea287c129ca0b3ae4d8be2bb00cfef22c60ff1852640ebc810a685",
    "8635d58d230e7d1cf4ba99b37328041a8be3c32c390458241121a39c79c95790",
    "d4d8e97fb33d005e1597286fbd1b6ad6f0c99e65d0fa55e4dce9e3e7f0739b",
    "09d353808c3bad9802747312c5422f4633642b944b3b9c022d28ecf9e43cd9c",
    "685b9f8174de14084aa97710a33ee73b903e108b3c4d1b506d32c8d2c472ad9d",
    "5f866c1abfe3a89add77d1aac6e0c7e73f78f52b3d8f6073bedabd91d6bd70a4",
    "70b7e1fb439dc5e324285c6284a9c1c20efca3b7c4161fddc6c1223a34cb835a5",
    "88587aa23ba7344e2c2ea1bfc246f9229b0d3b4fd953585d24ad8f1fee8408af",
    "09fd1fb895ed2deb63e9df3e12999dd1ec8a66355c834beee06664f4208067b6",
    "fff10595101811df8df7c4d56bb3fd7221b1717a77dd44f78c2200a1f0a188ba",
    "f1f441c9e48dc60c9aeb5e08315613cd26d4f19b13aae5d4f9a169543b4a1c6",
    "93870ecc3bd6236a51884c53255118163212e962b8df19dce65c6ffdd28cb79c8",
    "9ee949c43a6b95a9ab2c2df29f452375d14a4b67a4fb9c852fb48018d8c68d6",
    "8e139bcb278f628e1c4695ca70e46c5116dbbd63c5730a97b63b9fdb97e37b0b",
    "d30de5eb190aebae473df2f7c770d3a7dc7e5c38aced3733075b9ebd502de2",
    "fbb217555e0ded41187a16b4e54ff75e577e51d2e7026b13b06dd6749bca09e8",
    "00e8310f574c2362b3034672ad3d575837d1a6f690087fe8a2cc1ac1033eced",
    "6699aade9c937dc1b16500c70a0e5f2f07e1310d2c8fb49a1cadcf7f82f31ef2",
    "9e9b9f3fc7a0661055b8a4553fe70937b2b125d53751acc15e36a48c94ef2",
    "5a14345d5b0578044a0db1edb3b6faa5fa884db4c720daf59e6c5ede231072f3",
    "db24a785241d5ed73caea24da9549cfb12eac34dea9a5c9a460e50b0f51422",
    "430e89652a632ee9dcf3bdf5c612f637ba306ff244674f3adde8475a64773021",
    "2b55f3e3b58e16416547d734e0449b3b7f70971aa97573b4c9beb8a4327173",
    "bb5ef9ab88c18759d4fb5fab31ef3c58ed62ab267a4c338fd9f9fb2fdb03fb15",
    "180318c05418918b28f0a1a972ab13a71209676280fa9e51188df83b7f2a9c",
    "0c0833f633fa2fca726f2f0e96ea8ea029a3802f88b7b1c22bad7347cd98db",
    "c33498441a5c13c204033ad3ca370416c9e08b383363a8a56f35e6c014ec6ad",
    "b812118c2739421780e013c48cd3842137c9501df068af9c02d8b24500706ab8"
  ]
}

```

查询钱包余额，归还剩余的测试币，等待交易确认后，再次查询钱包余额，如图 11 所示，归还后钱包余额为 0.00000000 BTC。

图 11: 归还剩余测试币

```
> ~ bitcoin-cli -testnet getbalance
0.00141569
> ~ bitcoin-cli -testnet sendtoaddress tb1qerzrlxcfu24davlur5sqmgzzgsal6wusda40er 0.00141569 "" "" true
67a2397aeb5367f4ba18b73778cf83743e1770aa121b9bab887430d7fccfa55a
> ~ bitcoin-cli -testnet getbalance
0.00000000
```

3 实验结果

获取由 Bitcoin Core 客户端解析的测试交易与对应区块的数据，与自行解析的结果进行对比，两者关键字段一致，表明上述解析结果正确。

4 附录

本次实验过程中涉及到的地址、交易与区块信息可由下列网站获取：

- (1) 地址 [tb1qh8u89m0rkmzaqehwy6r208n9ur0q3wxcpe022d](#)
- (2) 地址 [tb1q42p7e3uqus54fxze9g26rha88nqpphlzumecua](#)
- (3) 地址 [tb1qc6g7mf96zzdyfgx6fz202gg03lxcg3znpgeenvn](#)
- (4) 交易 [c8887df396045d8fdee1e551fabfe2ab3e0b907c8f2ee2d5469d5c103c531039](#)
- (5) 交易 [bb5ef9ab88c18759d4fb5fab31ef3c58ed62ab267aba338fd9effb2fdb03fb15](#)
- (6) 交易 [67a2397aeb5367f4ba18b73778cf83743e1770aa121b9bab887430d7fccfa55a](#)
- (7) 区块 [5075657](#)
- (8) 区块 [5075763](#)
- (9) 区块 [5080294](#)

本次实验的代码、数据与解析结果文件均已以附件形式上传。